

SCJP 1.6 (CX-310-065 , CX-310-066)

Subject: Operators

Total Questions : 16

Prepared by : <http://www.javacertifications.net>

SCJP 6.0: Operator

Questions Question - 1

What is the output for the below code ?

```
1. public class Test {
2.     public static void main(String... args) {
3.         int x =5;
4.         x *= 3 + 7;
5.         System.out.println(x);
6.     }
7. }
```

1.22

2.50

3.10

4.Compilation fails with an error at line 4

Explanation :

B is the correct answer.

Shot handle operations i.e

$x = 5 \implies x = x - 5$

$x *= 3 + 7$; is same as $x = x * (3 + 7) = 5 * (10) = 50$ because expression on the right is always placed inside parentheses.

Question - 2

What is the output for the below code ?

```
1. public class Test {
2.     public static void main(String... args) {
3.         System.out.println('a' == 'a');
4.         System.out.println('5' == '5');
5.         System.out.println(7 != 8);
6.         System.out.println(7.0 == 7L);
7.     }
8. }
```

1.true true false true

2.true true true true

3.Compilation fails with an error at line 4

4.Compilation fails with an error at line 6

Explanation :

B is the correct answer.

'a' == 'a' returns true. '5' == '5' returns true. 7 != 8 returns true because 7 is not equal to 8. 7.0 == 7L returns true.

Question - 3

What is the output for the below code ?

```
1. public class Test {  
2.     public static void main(String... args) {  
3.         Integer int5 = new Integer(5);  
4.         System.out.println(int5 == 5);  
5.         System.out.println(int5 == '5');  
6.     }  
7. }
```

- 1.true true
- 2.true false
- 3.Compilation fails with an error at line 4
- 4.Compilation fails with an error at line 5

Explanation :

B is the correct answer.

From j2se 5.0 onwards you can compare Integer Object and int primitive type. So int5 == 5 return true. But int5 == '5' return false because '5' is char.

Question - 4

What is the output for the below code ?

```
1. import java.awt.Button;  
2. public class Test {  
3.     public static void main(String... args) {  
4.         Button b = new Button("submit");  
5.         Button a = b;  
6.         Button c = a;  
7.         System.out.println(a == c);  
8.     }  
9. }
```

- 1.true
- 2.false
- 3.Compilation fails with an error at line 5
- 4.Compilation fails with an error at line 6

Explanation :

A is the correct answer.

Both reference variables a and c refer to the same object. So a == c returns true.

Question - 5

What is the output for the below code ?

```
1. public class Test {  
2.     public static void main(String... args) {  
3.         String s1 = new String("hello");  
4.         String s2 = new String("hello");  
5.         System.out.println(s1 == s2);  
6.         System.out.println(s1.equals(s2));  
7.     }  
8. }
```

- 1.true false
- 2.false true
- 3.false false
- 4.Compilation fails with an error at line 6

Explanation :

B is the correct answer.

Both reference variables s1 and s2 refer to the different String object. So s1 == s2 returns false. But s1 refer to a String object which contain "hello" and s2 also refer to a String object which contain "hello" So s1.equals(s2) returns true. equals method of String class checks for the contents.

Question - 6

What is the output for the below code ?

```
1. public class Test {  
2.     enum Month { JAN, FEB, MAR };  
3.     public static void main(String... args) {  
4.         Month m1 = Month.JAN;  
5.         Month m2 = Month.JAN;  
6.         Month m3 = Month.FEB;  
7.         System.out.println(m1 == m2);  
8.         System.out.println(m1.equals(m2));  
9.         System.out.println(m1 == m3);  
10.        System.out.println(m1.equals(m3));  
11.    }  
12. }
```

- 1.true true true false
- 2.true true false false
- 3.false false true true
- 4.Compilation fails with an error at line 10

Explanation :

B is the correct answer.

m1 and m2 refer to the same enum constant So m1 == m2 returns true BUT m1 and m3 refer to different enum constant So m1 == m3 returns false. m1.equals(m2) returns true because enum constant value is same (JAN and JAN). m1.equals(m3) return false because enum constants values are different (JAN and FEB).

Question - 7

What is the output for the below code ?

```
public class A {
    public void printValue() {
        System.out.println("A");
    }
}

public class B extends A {
    public void printValue() {
        System.out.println("B");
    }
}

public class C extends A {
    public void printValue() {
        System.out.println("C");
    }
}

1. public class Test {
2.     public static void main(String... args) {
3.         A b = new B();
4.         A c = new C();
5.         System.out.println(b instanceof A);
6.         System.out.println(b instanceof C);
7.     }
8. }
```

1. true true
2. true false
3. false false
4. Compilation fails with an error at line 5 and 6

Explanation :

B is the correct answer.

instanceof operator is used for object reference variables to check whether an object is of a particular type. reference variable b is a type of A BUT b is not type C.

Question - 8

What is the output for the below code ?

```
public class A {
```

```

        public void printValue() {
            System.out.println("A");
        }
    }

    public class B extends A {
        public void printValue() {
            System.out.println("B");
        }
    }

1. public class Test {
2.     public static void main(String... args) {
3.         A b = new B();
4.         newValue(b);
5.     }
6.     public static void newValue(A a) {
7.         if(a instanceof B) {
8.             ((B)a).printValue();
9.         }
10.    }
11. }

```

- 1.A
- 2.B
- 3.Compilation fails with an error at line 4
- 4.Compilation fails with an error at line 8

Explanation :

B is the correct answer.

instanceof operator is used for object reference variables to check whether an object is of a particular type. In newValue(b); b is instance of B So works properly.

Question - 9

What is the output for the below code ?

```

public class A {
    public void printValue() {
        System.out.println("A");
    }
}

public class B extends A {
    public void printValue() {
        System.out.println("B");
    }
}

public class C extends B {
    public void printValue() {
        System.out.println("C");
    }
}

```

```

public class Test {
    public static void main(String... args) {
        A a = new A();
        B b = new B();
        C c = new C();
        System.out.println(b instanceof A);
        System.out.println(c instanceof A);
    }
}

```

- 1.true false
- 2.true true
- 3.false false
- 4.false true

Explanation :

B is the correct answer.

instanceof operator is used for object reference variables to check whether an object is of a particular type. b and c reference variable are type of class A because B extends A and then C extends B.

Question - 10

What is the output for the below code ?

```

interface A {
}

public class B implements A {
    public void printValue() {
        System.out.println("B");
    }
}

public class C extends B{
    public void printValue() {
        System.out.println("C");
    }
}

public class Test {
    public static void main(String... args) {
        B b = new B();
        C c = new C();
        System.out.println(b instanceof A);
        System.out.println(c instanceof A);
    }
}

```

- 1.true false
- 2.true true
- 3.false false
- 4.false true

Explanation :

B is the correct answer.

instanceof operator is used for object reference variables to check whether an object is of a particular type. b and c reference variable are type of interface A because B implements A and then C extends B.

Question - 11

What is the output for the below code ?

```
public class Test {  
    public static void main(String... args) {  
        System.out.println(null instanceof Object);  
        System.out.println(null instanceof String);  
    }  
}
```

- 1.true false
- 2.false false
- 3.true true
- 4.false true

Explanation :

B is the correct answer.

null is not instance of Object or String.

Question - 12

What is the output for the below code ?

```
1. public class Test {  
2.     public static void main(String... args) {  
3.         int [] index = new int[5];  
4.         System.out.println(index instanceof Object);  
5.         System.out.println(index instanceof String);  
6.     }  
7. }
```

- 1.true false
- 2.false false
- 3.true true
- 4.Compilation fails with an error at line 5

Explanation :

D is the correct answer.

Incompatible conditional operand types int[] and String. There is no way index could ever refer to a String type.

Question - 13

What is the output for the below code ?

```
1. public class Test {  
2.     public static void main(String... args) {  
3.         int [] index = new int[5];  
4.         System.out.println(index instanceof Object);  
5.     }  
6. }
```

- 1.true
- 2.false
- 3.Compilation fails with an error at line 3
- 4.Compilation fails with an error at line 4

Explanation :

A is the correct answer.

An array is always an instance of Object

Question - 14

What is the output for the below code ?

```
public class Test {  
    public static void main(String... args) {  
        int a =5 , b=6, c =7;  
        System.out.println("Value is "+ b +c);  
        System.out.println(a + b +c);  
        System.out.println("String "+(b+c));  
    }  
}
```

- 1.Value is 67 18 String 13
- 2.Value is 13 18 String 13
- 3.Value is 13 18 String
- 4.Compilation fails

Explanation :

A is the correct answer.

If the left hand operand is not a String then + operator treat as plus BUT if left hand operand is a String then + perform String concatenation.

Question - 15

What is the output for the below code ?

```
1. public class Test {
2.     static int i =5;
3.     public static void main(String... args) {
4.         System.out.println(i++);
5.         System.out.println(i);
6.         System.out.println(++i);
7.         System.out.println(++i+i++);
8.
9.     }
10. }
```

1.5 6 7 16

2.6 6 6 16

3.6 6 7 16

4.5 6 6 16

Explanation :

A is the correct answer.

i++ : print value then increment (postfix - increment happens after the value of the variable is used) ++i : increment the print (prefix - increment happens before the value of the variable is used)

Question - 16

What is the output for the below code ?

```
1. public class Test {
2.     public static void main(String... args) {
3.         Integer i = 34;
4.         String str = (i<21)?"jan":(i<56)?"feb":"march";
5.         System.out.println(str);
6.     }
7. }
```

1.feb

2.jan

3.march

4.Compilation fails with an error at line 4

Explanation :

A is the correct answer.

This is nested conditional with unbox. (i<21) is false goto (i<56), (i<56) is true so result is "feb".

